

Predicting Daily Land Average Temperature

Summary

This project analyzes global daily average land-surface temperature measurements collected by Berkeley Earth from 1880–2022. In the following analysis, dataset is cleaned and preprocessed, explored through EDA, and modeled to predict future temperature trends using this historical data. After evaluating three regression approaches, Linear Regression, Random Forest, and Support Vector Regression (SVR), the SVR model performed best and was used to forecast the land average temperature for the year 2030.

Introduction

Understanding long-term changes in global land temperature is critical for studying climate change. Daily temperature anomaly data collected over more than 140 years provide an opportunity to model how temperatures have shifted, identify long-term patterns, and forecast future warming.

In this report, we talk both about the actual temperature as well as the temperature anomaly. Temperature anomaly refers to deviation from a baseline climatological temperature. In this study, the baseline is 8.59°C, representing the January 1951–December 1980 global land-average temperature.

In this project, we will be answering the research question:

What do we expect the global land-average temperature of the Earth to be in 2030, based on the trends from the years 1880 to 2012.

Dataset

The data set for this project was published by Berkeley Earth under a Creative Commons BY-NC 4.0 International license, free for non-commercial use, and accessed by our team compliant with the conditions in this license on November 18, 2025. The raw data can be found at https://berkeley-earth-temperature.s3.us-west-1.amazonaws.com/Global/Complete_TAVG_daily.txt.

The data set contains 5 columns with time series information, and one column representing the temperature difference relative to the average temperature between January 1951 and December 1980, which they calculated as 8.59 +/- 0.05. For our

analysis, we preprocessed the data to get the raw temperature readings back by adding 8.59 to each entry in the Anomaly column. All temperatures are in Celcius.

Methods and Results

Importing the required libraries. Execution Time Note: It may take up to 30 seconds to load in the libraries on the first run through.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import altair as alt
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import warnings

# Simplify Working with Large Datasets
alt.data_transformers.enable('vegafusion')

# Ignore all warnings from the altair package for pdf rendering, warnings via
# resource for implementation:
# https://stackoverflow.com/questions/3920502/how-to-suppress-a-third-party-
warnings.filterwarnings('ignore', module='altair')

# Configure Plot Sizes for pdf (D.R.Y)
plot_size = {'width': 450, 'height': 300}
facet_plot_size = {'width': 250, 'height': 200}
```

Load the Data

Load the daily land-surface average anomaly data provided by Berkeley Earth collected from 1880 to 2022.

```
In [2]: url = "https://berkeley-earth-temperature.s3.us-west-1.amazonaws.com/Global/
...
read the data from the url link
ignore the comments starting with '%'
ignore the header in the comments and assign manually
...
df = pd.read_csv(url, sep=r"\s+", comment="%", header=None)

# assign column headers
column_names = ["Date Number", "Year", "Month", "Day", "Day of Year", "Anoma
df.columns = column_names
```

```
#df.to_csv("data/raw.csv", index=False)
```

Data Pre-Processing

```
In [3]: df = df.drop(columns=['Date Number'])
```

```
In [4]: BASELINE_TEMP = 8.59 # Jan 1951-Dec 1980 land-average temperature in celsius  
df['Temperature'] = df['Anomaly'] + BASELINE_TEMP
```

```
In [5]: month_dict = {  
    1: 'January',  
    2: 'February',  
    3: 'March',  
    4: 'April',  
    5: 'May',  
    6: 'June',  
    7: 'July',  
    8: 'August',  
    9: 'September',  
   10: 'October',  
   11: 'November',  
   12: 'December'  
}  
  
df['Month_Name'] = df['Month'].map(month_dict)
```

```
In [6]: # Order the month names as categorical  
df['Month_Name'] = pd.Categorical(df['Month_Name'], categories=list(month_dict.values()))
```

Data Splitting

```
In [7]: # Split into training and test dataframes based on decided cutoff  
cutoff = 2012  
test_df = df[df['Year'] > cutoff]  
train_df = df[df['Year'] <= cutoff]
```

Out [7]:

	Year	Month	Day	Day of Year	Anomaly	Temperature	Month_Name
0	1880	1	1	1	-0.692	7.898	January
1	1880	1	2	2	-0.592	7.998	January
2	1880	1	3	3	-0.673	7.917	January
3	1880	1	4	4	-0.615	7.975	January
4	1880	1	5	5	-0.681	7.909	January
...	...	...	...	...	...	...	...
48573	2012	12	27	361	0.134	8.724	December
48574	2012	12	28	362	0.371	8.961	December
48575	2012	12	29	363	0.532	9.122	December
48576	2012	12	30	364	0.594	9.184	December
48577	2012	12	31	365	0.631	9.221	December

48578 rows x 7 columns

Data Validation

Validation Checklist:

- [] 7. No outlier or anomalous values - before split
- [] 8. Correct category levels (i.e., no string mismatches or single values) - before split
- [] 9. Target/response variable follows expected distribution - after split
- [] 10. No anomalous correlations between target/response variable and features/explanatory variables - after split
- [] 11. No anomalous correlations between features/explanatory variables - after split
- **Perform validation first with pandas then translate to pandera schemas**

```
In [8]: # Schema Validation import
import pandera as pa
```

- [] 7. No outlier or anomalous values
 - Visualization Methods for detecting numerical outliers
 - Quantitative Statistics for detecting numerical outliers
 - Analyze target and features "outliers".

- Write validation schema for expected ranges
- Test validation schema

In [9]: *#region Pseudo Code:*

```
# use train dataset first but may need to check test set too though
# that indicate invalid example due to unreasonable value
# plot anomaly and temperature for numerical outliers,
# perhaps boxplot across years or mean per year, use pandera if applicable
# calculate variance, quantiles explicitly etc. use pandera if applicable
# analyze or plot unique categories (numbers except for month_name)
# define reasonable limits on data in a schema

#endregion

# visualize outliers - entire dataset

temp_points = alt.Chart(train_df,
    title=alt.Title(
        text='Global Daily Average Land Temperature (Figure 0_1)',
        subtitle='Temperature (Blue), Anomaly (Orange)')
).mark_point(size=1, opacity=0.6).encode(
    x = alt.X('Year:T', title='Year'),
    y = alt.Y('Temperature:Q',
        title='Temperature [°C]').scale(zero=False)
)

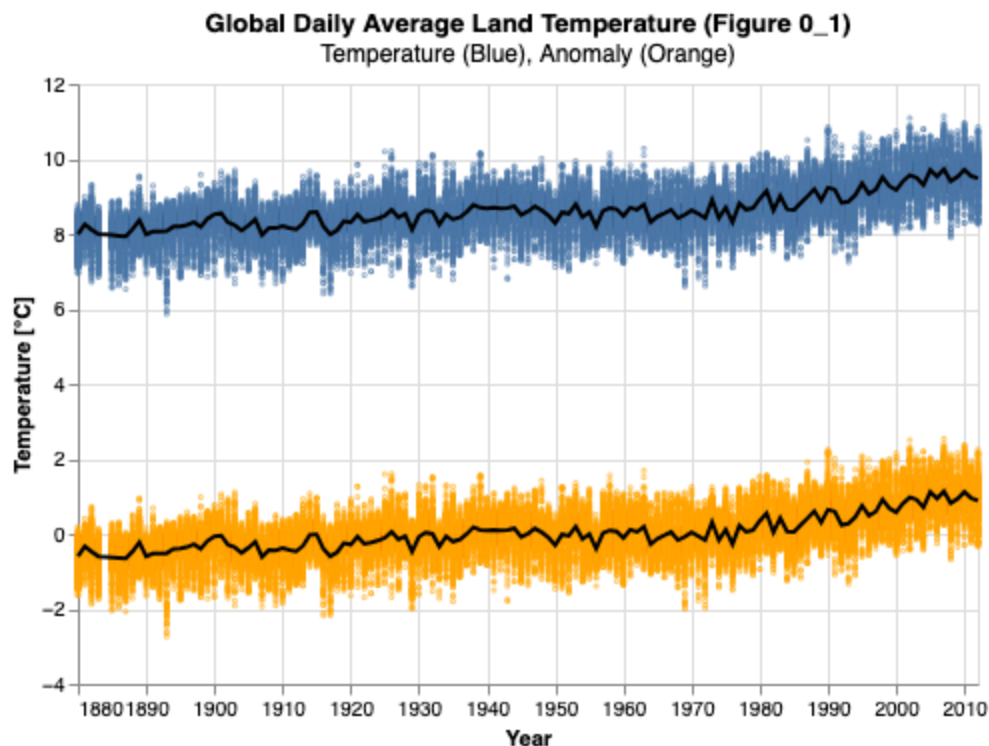
anom_points = alt.Chart(train_df,
    title=alt.Title(
        text='Annual Means of Global Daily Average Land Temperature (Figure
        subtitle='Temperature (Blue), Anomaly (Orange)')
).mark_point(size=1, color='orange', opacity=0.6).encode(
    x = alt.X('Year:T', title='Year'),
    y = alt.Y('Anomaly:Q',
        title='Temperature [°C]').scale(zero=False)
)

tp_avg = temp_points.mark_line(color='black').encode(
    x = alt.X('Year:T', title='Year'),
    y = alt.Y('mean(Temperature):Q',
        title='Temperature [°C]').scale(zero=False)
)

ap_avg = anom_points.mark_line(color='black').encode(
    x = alt.X('Year:T', title='Year'),
    y = alt.Y('mean(Anomaly):Q',
        title='Temperature [°C]').scale(zero=False)
)

comb = temp_points+anom_points+tp_avg+ap_avg
comb.properties(**plot_size)
```

Out [9]:



```
In [10]: train_df.describe(include='all')
```

```
Out[10]:
```

	Year	Month	Day	Day of Year	Anomaly	1
count	48578.000000	48578.000000	48578.000000	48578.000000	48578.000000	48
unique	NaN	NaN	NaN	NaN	NaN	
top	NaN	NaN	NaN	NaN	NaN	
freq	NaN	NaN	NaN	NaN	NaN	
mean	1946.000947	6.522953	15.729569	183.000000	0.016194	
std	38.393532	3.448732	8.800154	105.331318	0.607513	
min	1880.000000	1.000000	1.000000	1.000000	-2.728000	
25%	1913.000000	4.000000	8.000000	92.000000	-0.367000	
50%	1946.000000	7.000000	16.000000	183.000000	-0.034000	
75%	1979.000000	10.000000	23.000000	274.000000	0.372750	
max	2012.000000	12.000000	31.000000	365.000000	2.552000	

```
In [11]: print('train_df Anomaly sample median:', train_df['Anomaly'].median())
print('train_df Temperature sample median:', train_df['Temperature'].median())
print('train_df Anomaly sample standard deviation:', train_df['Anomaly'].std())
print('train_df Temperature sample standard deviation:', train_df['Temperature'].std())
```

```
train_df Anomaly sample median: -0.034
train_df Temperature sample median: 8.556
train_df Anomaly sample standard deviation: 0.6075125667255499
train_df Temperature sample standard deviation: 0.6075125667255499
```

```

In [12]: # Adapted from https://github.com/skysheng7/DSCI522_data_validation_demo/tree/master/region_old_analysis

#region old analysis
# # Guidance on outliers 3*sigma cutoff here:
# # https://www.scribbr.com/statistics/outliers/
# sdev_anomaly = train_df['Anomaly'].std() # calculate stat on train_df not
# # based on train_df analysis above set 3*sigma limit for outliers (99.7% c

# anomaly_sigma_limits = {'low_limit': -3*sdev_anomaly, 'high_limit': 3*sdev

# anomaly_mean_per_year = train_df.groupby(['Year'])['Anomaly'].mean().reset

# anomaly_mean_observed_limits = {'low_limit': min(anomaly_mean_per_year['Ar
#                                     'high_limit': max(anomaly_mean_per_year['
#endregion

import logging
import json
import warnings

# suppress verbose pandera warning for ipynb and pdf rendering
warnings.filterwarnings('ignore', module='pandera')

# Configure logging
logging.basicConfig(
    filename="validation_errors.log",
    filemode="w",
    format="%(asctime)s - %(message)s",
    level=logging.INFO,
)

# Correct Data Collection Limits for Target and Transformed Target
deviation_limit = 10
anomaly_overall_limits = {'low_limit': -deviation_limit,
                           'high_limit': deviation_limit}

# Limits this high should never be practically seen in the date range
# Temperature Limits Depend on anomaly limits since temperature is just
# anomaly plus a constant

# validate data
target_schema = pa.DataFrameSchema(
    {
        "Temperature": pa.Column(float, pa.Check.between(
            anomaly_overall_limits["low_limit"]+BASELINE_TEMP,
            anomaly_overall_limits["high_limit"]+BASELINE_TEMP), nullable=False),
        # Do not allow nulls
        "Anomaly": pa.Column(float, pa.Check.between(
            anomaly_overall_limits["low_limit"],
            anomaly_overall_limits["high_limit"]), nullable=False),
        # Do not allow nulls
    },
    checks=[
        pa.Check(lambda df: ~df.duplicated().any(), error="Duplicate rows fo
        pa.Check(lambda df: ~(df.isna().all(axis=1)).any(), error="Empty row
    ],

```

```

        drop_invalid_rows=False,

    )

    # Validate the Schema for Anomaly and Temperature Column
    val_df = target_schema.validate(df, lazy=True)

    # test schema with data points outside limits
    try:
        df_test = df[['Anomaly', 'Temperature']].add(deviation_limit, axis=0)
        target_schema.validate(df_test, lazy=True)

    except pa.errors.SchemaErrors as e:
        print('\nValidation Test DataFrame Schema Error Thrown:\n')
        print('Error Type: ', type(e))
        print('Error Name: ', type(e).__name__)
        print('Error Counts:\n', e.error_counts)
        print('Failure Cases:\n', e.failure_cases)
        error_cases = e.failure_cases
        # Convert the error message to a JSON string
        error_message = json.dumps(e.message, indent=2)
        logging.error("\n" + error_message)

        # Inspect error object attributes for better displaying
        # print(e.__dir__())

    # Check if rows dropped
    # print(f'\nlen(df) == len(val_df) -> {len(df) == len(val_df)}:',
    #       'No Rows Dropped if "True"')

    # Show Validated df - not sure if printing the df counts as "peeking at the
    # val_df

```


Validation Test DataFrame Schema Error Thrown:

Error Type: <class 'pandera.errors.SchemaErrors'>

Error Name: SchemaErrors

Error Counts:

defaultdict(<class 'int'>, {'DATAFRAME_CHECK': 3})

Failure Cases:

	schema_context	column	check
\			
52904	DataFrameSchema	None	Duplicate rows found.
35273	Column	Anomaly	in_range(-10, 10)
35263	Column	Anomaly	in_range(-10, 10)
35264	Column	Anomaly	in_range(-10, 10)
35265	Column	Anomaly	in_range(-10, 10)
...	...	...	...
17639	Column	Temperature	in_range(-1.4100000000000001, 18.59)
17640	Column	Temperature	in_range(-1.4100000000000001, 18.59)
17641	Column	Temperature	in_range(-1.4100000000000001, 18.59)
17642	Column	Temperature	in_range(-1.4100000000000001, 18.59)
26452	Column	Anomaly	in_range(-10, 10)

	check_number	failure_case	index
52904	0	0.000	None
35273	0	10.189	28991
35263	0	10.097	28976
35264	0	10.036	28979
35265	0	10.373	28980
...	...	...	...
17639	0	19.025	43036
17640	0	19.132	43037
17641	0	19.076	43038
17642	0	19.292	43039
26452	0	10.043	311

[52905 rows x 6 columns]

```
In [13]: years_selection = [1880, 1920, 1960, 2000, cutoff]

box = alt.Chart(train_df[train_df['Year'].isin(years_selection)]).mark_boxplot
x = 'Year:N',
y = alt.Y('Temperature', title='Temperature [°C]').scale(zero=False),
color = 'Year:N'
)
box_point = alt.Chart(train_df[train_df['Year'].isin(years_selection)]).mark
x = 'Year:N',
y = alt.Y('mean(Temperature)').scale(zero=False),
color = alt.value('black')
)

comb1 = box+box_point
comb1 = comb1.properties(**plot_size,
                          title = 'Distributions of Global Daily Average Land Temperat

box = alt.Chart(train_df[train_df['Year'].isin(years_selection)]).mark_boxplot
x = 'Year:N',
y = alt.Y('Anomaly', title='Anomaly [°C]').scale(zero=False),
```

```

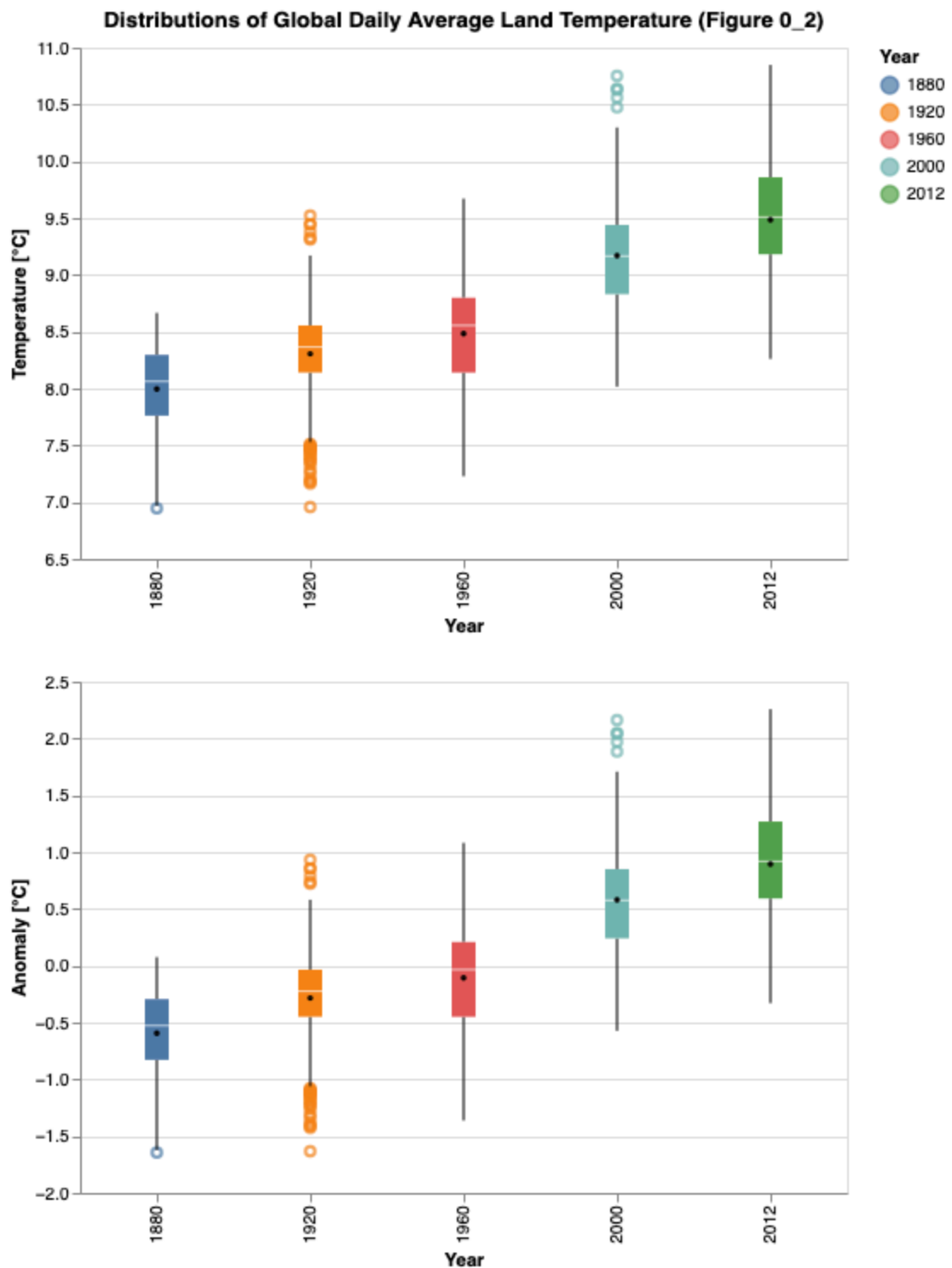
        color = 'Year:N'
    )
box_point = alt.Chart(train_df[train_df['Year'].isin(years_selection)]).mark
    x = 'Year:N',
    y = alt.Y('mean(Anomaly)').scale(zero=False),
    color = alt.value('black')
)

comb2 = box+box_point
comb2 = comb2.properties(**plot_size,
)

comb1 & comb2

```

Out[13]:



- [] 7. Continued For Features...
- [] 8. Correct category levels (i.e., no string mismatches or single values)
 - check Month is 1-12, and only 1-12, day 1-31, day of year 1-365 and month_name is January-December (all inclusive)
 - check correct unique values and no single values
 - this step may be safe to do on full data set
 - Setup validation schema

- Test validation schema

In [14]: *#region Pseudo Code:*

```
# do on train dataset first to be safe add test data set later if confirmed
# pandas version of checking categories – similar checks done in preprocessi
# translate to robust panders schemas

#endregion

# Adapted from https://github.com/skysheng7/DSCI522_data_validation_demo/tree

# Correct Data Collection Limits for Features
year_limits = {'low_limit': 1880, 'high_limit': 2022}
month_limits = {'low_limit': 1, 'high_limit': 12}
doy_limits = {'low_limit': 1, 'high_limit': 365}
day_limits = {'low_limit': 1, 'high_limit': 31}
# Visibility
# print('pa.Check.isin() argument:\n', list(month_dict.values()))

# validate data
features_schema = pa.DataFrameSchema(
    {
        "Year": pa.Column(int, pa.Check.between(
            year_limits["low_limit"],
            year_limits["high_limit"]), nullable=False),
        "Month": pa.Column(int, pa.Check.between(
            month_limits["low_limit"],
            month_limits["high_limit"]), nullable=False),
        "Day": pa.Column(int, pa.Check.between(
            day_limits["low_limit"],
            day_limits["high_limit"]), nullable=False),
        "Day of Year": pa.Column(int, pa.Check.between(
            doy_limits["low_limit"],
            doy_limits["high_limit"]), nullable=False),
        "Month_Name": pa.Column(str, pa.Check.isin(
            list(month_dict.values()))), nullable=False)
        # Do not allow nulls for any
    },
    checks=[
        pa.Check(lambda df: ~df.duplicated().any(), error="Duplicate rows fo
        pa.Check(lambda df: ~(df.isna().all(axis=1)).any(), error="Empty row
    ],
    drop_invalid_rows=False,
)

# Validate the Schema for Anomaly and Temperature Column
val_df = features_schema.validate(df, lazy=True)

# test schema with data points outside limits
try:
    df_test = df[['Year', 'Month', 'Day', 'Day of Year']].add(1, axis=0)
    df_test['Month_Name'] = 'Feb'
```

```

# print(df_test['Day'])
features_schema.validate(df_test, lazy=True)

except pa.errors.SchemaErrors as e:
    print('\nValidation Test DataFrame Schema Error Thrown:\n')
    print('Error Type: ', type(e))
    print('Error Name: ', type(e).__name__)
    print('Error Counts:\n', e.error_counts)
    print('Failure Cases:\n', e.failure_cases)
    error_cases = e.failure_cases
    # Convert the error message to a JSON string
    error_message = json.dumps(e.message, indent=2)
    logging.error("\n" + error_message)

# Check if rows dropped
# print(f'\nlen(df) == len(val_df) -> {len(df) == len(val_df)}:',
#       'No Rows Dropped if "True"')

# Show Validated df - not sure if printing the df counts as "peeking at the
# val_df

```

Validation Test DataFrame Schema Error Thrown:

Error Type: <class 'pandera.errors.SchemaErrors'>

Error Name: SchemaErrors

Error Counts:

defaultdict(<class 'int'>, {'DATAFRAME_CHECK': 5})

Failure Cases:

	schema_context	column \
0	Column	Year
38517	Column	Month_Name
38547	Column	Month_Name
38548	Column	Month_Name
38549	Column	Month_Name
...	...	...
19281	Column	Month_Name
19282	Column	Month_Name
19283	Column	Month_Name
19284	Column	Month_Name
57830	Column	Month_Name

	check	check_number \
0	in_range(1880, 2022)	0
38517	isin(['January', 'February', 'March', 'April',...]	0
38547	isin(['January', 'February', 'March', 'April',...]	0
38548	isin(['January', 'February', 'March', 'April',...]	0
38549	isin(['January', 'February', 'March', 'April',...]	0
...	...	...
19281	isin(['January', 'February', 'March', 'April',...]	0
19282	isin(['January', 'February', 'March', 'April',...]	0
19283	isin(['January', 'February', 'March', 'April',...]	0
19284	isin(['January', 'February', 'March', 'April',...]	0
57830	isin(['January', 'February', 'March', 'April',...]	0

	failure_case	index
0	2023	51865
38517	Feb	32763
38547	Feb	32793
38548	Feb	32794
38549	Feb	32795
...	...	...
19281	Feb	13527
19282	Feb	13528
19283	Feb	13529
19284	Feb	13530
57830	Feb	52076

[57831 rows x 6 columns]

- [] 9. Target/response variable follows expected distribution
 - check skewness, visualize distribution
 - show anomaly and temperature for easier interpretation
 - add validation check for a property of the distribution that should hold
 - Setup validation schema
 - Test validation schema

In [15]: *#region Pseudo Code:*

```
# definitely only use train dataset
# show skewness
# visualization of target via histogram or density plot or both
# quantitative evaluation using mean std, var, quantiles etc.
# translate to robust panders schemas

#endregion

# https://altair-viz.github.io/user_guide/marks/rule.html
# 531 Lecture 6 Notes Adaptations

# Visualization only on train_df

temp_hist = alt.Chart(train_df).mark_bar().encode(
    x = alt.X('Temperature:Q', bin=alt.Bin(maxbins=50), title='Temperature |
    y=alt.Y('count()')
    # mark rule was messing up scale otherwise
).properties(**plot_size, title = 'Target Distrubtions of the Training Datas

temp_mean = alt.Chart(train_df).mark_rule(color="red").encode(
    x="mean(Temperature):Q",
    size=alt.value(2),
)

temp_med = alt.Chart(train_df).mark_rule(color="black").encode(
    x="median(Temperature):Q",
    size=alt.value(2),
)

temp_hist = temp_hist+temp_med+temp_mean

anomaly_hist = alt.Chart(train_df).mark_bar(color='orange').encode(
    x = alt.X('Anomaly:Q', bin=alt.Bin(maxbins=50), title='Anomaly [°C]'),
    y=alt.Y('count()')
).properties(**plot_size)

anom_mean = alt.Chart(train_df).mark_rule(color="red").encode(
    x="mean(Anomaly):Q",
    size=alt.value(2),
)

anom_med = alt.Chart(train_df).mark_rule(color="black").encode(
    x="median(Anomaly):Q",
    size=alt.value(2),
)

anomaly_hist = anomaly_hist+anom_med+anom_mean

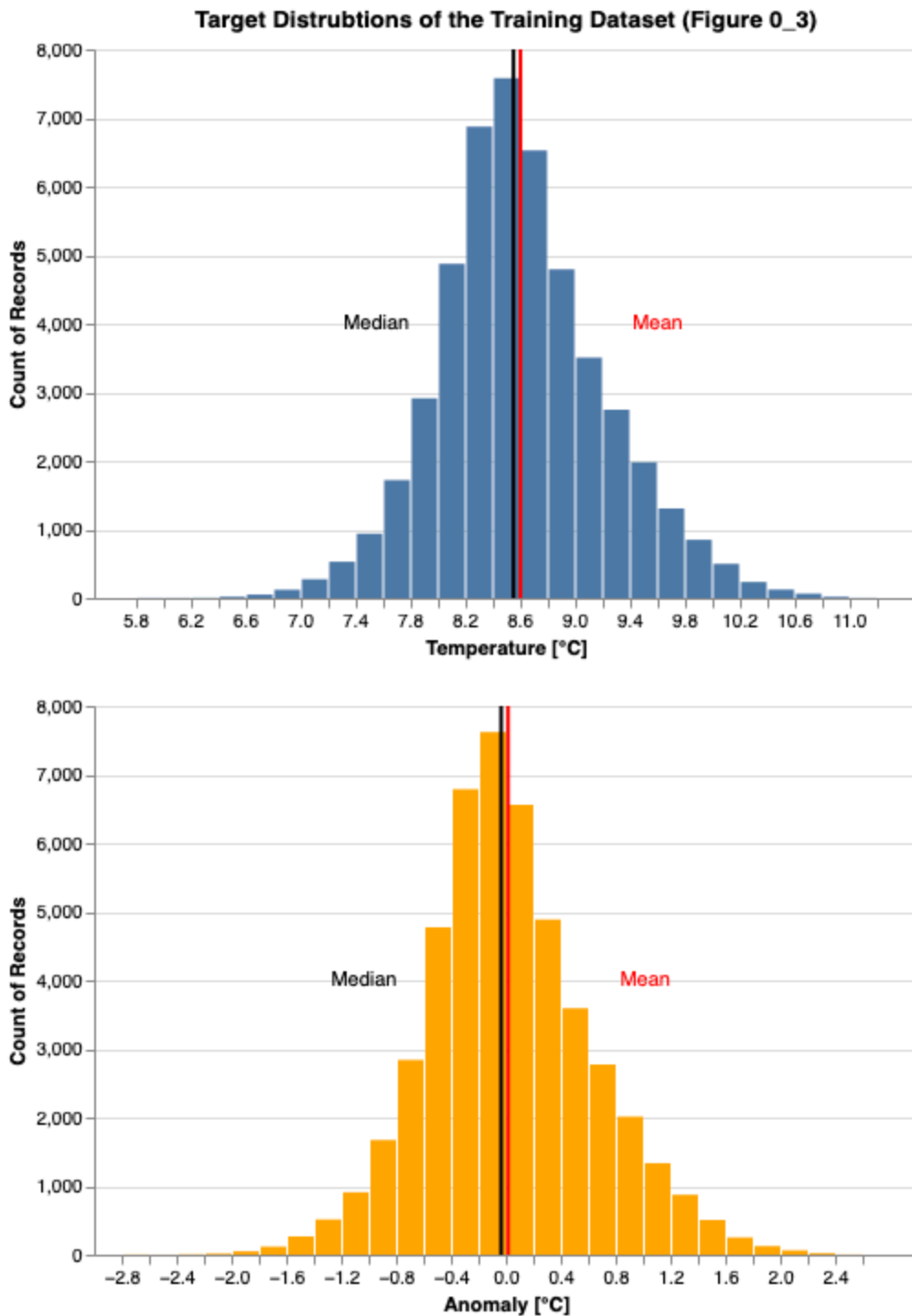
plot = (temp_hist + alt.Chart(train_df).mark_text(dx=-75).encode(
    x="median(Temperature):Q",
    text=alt.value('Median'))+ alt.Chart(train_df).mark_text(
        dx=75, color='red').encode(
    x="mean(Temperature):Q",
```

```

text=alt.value('Mean'))
) & (anomaly_hist + alt.Chart(train_df).mark_text(dx=-75).encode(
  x="median(Anomaly):Q",
  text=alt.value('Median')) + alt.Chart(train_df).mark_text(
    dx=75, color='red').encode(
  x="mean(Anomaly):Q",
  text=alt.value('Mean'))))
plot

```

Out[15]:



We can see from the histogram plots of the target (Anomaly untransformed and Temperature when transformed) that the distribution of the target is very slightly right-skewed, but is almost symmetric and fairly bell-shaped. We would expect the


```

print('\nValidation Test DataFrame Schema Error Thrown:\n')
print('Error Type: ', type(e))
print('Error Name: ', type(e).__name__)
print('Error Counts:\n', e.error_counts)
print('Failure Cases:\n', e.failure_cases)
error_cases = e.failure_cases
# Convert the error message to a JSON string
error_message = json.dumps(e.message, indent=2)
logging.error("\n" + error_message)

# Check if rows dropped
# print(f'\nlen(df) == len(val_df) -> {len(df) == len(val_df)}:',
#       'No Rows Dropped if "True"')

# Show Validated df - not sure if printing the df counts as "peeking at the
# val_df

```

```

df_test means:  Anomaly      -1.143347
Temperature    2.207529
dtype: float64

```

```

df_test medians: Anomaly      -0.908819
Temperature     2.196446
dtype: float64

```

Validation Test DataFrame Schema Error Thrown:

```

Error Type:  <class 'pandera.errors.SchemaErrors'>
Error Name:  SchemaErrors
Error Counts:
  defaultdict(<class 'int'>, {'DATAFRAME_CHECK': 2})
Failure Cases:
   column  failure_case index  schema_context \
0   None           False  None  DataFrameSchema
1   None           False  None  DataFrameSchema

                                check  check_number
0                Duplicate rows found.           0
1  Anomaly Distribution Left-skewed           3

```

- [] 10. No anomalous correlations between target/response variable and features/explanatory variables

In [17]: *#region Pseudo Code:*

```

# definitely only use train dataset
# check correlations between train_df columns and temp/anomaly
# expectation is that target is correlated to some degree with time features

#endregion

# Adapted from https://github.com/skysheng7/DSCI522_data_validation_demo/tree/master
# Correlation with Features and Target, train_df

```

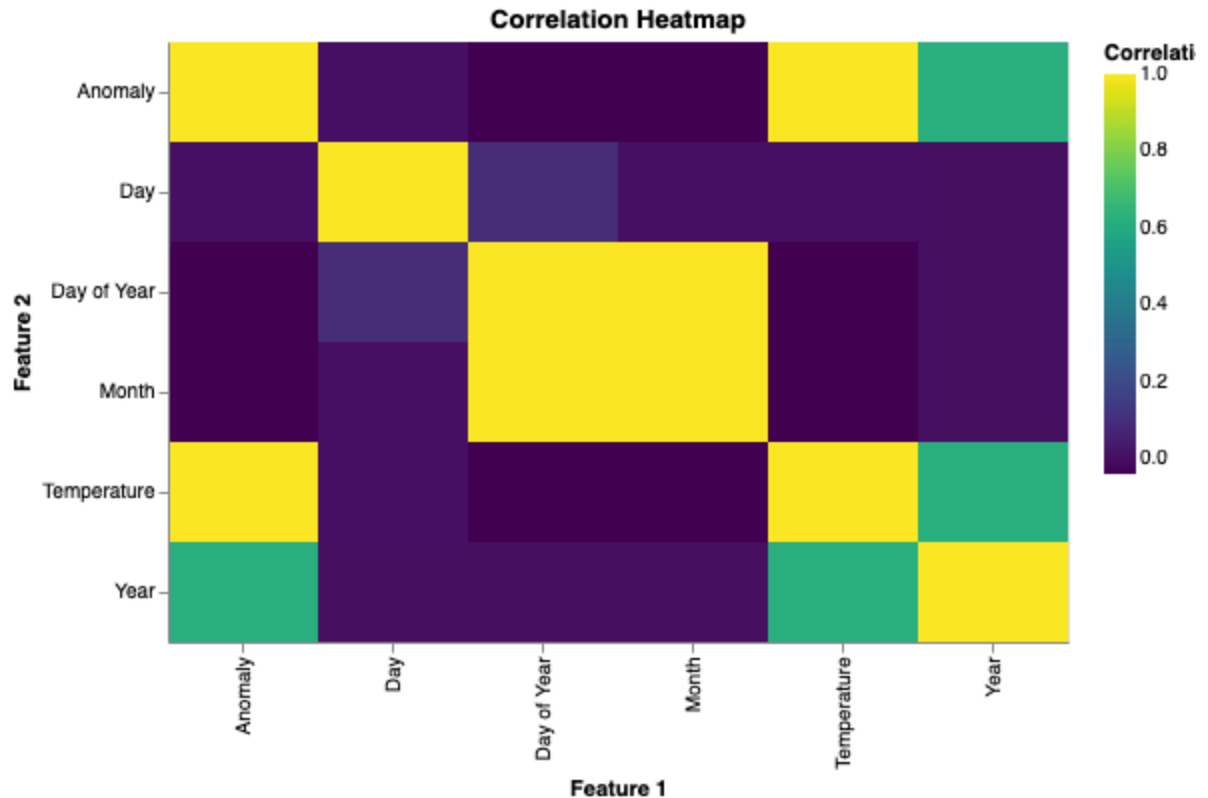
```

correlation_matrix = train_df.drop(columns=['Month_Name']).corr()
correlation_long = correlation_matrix.reset_index().melt(id_vars='index')
correlation_long.columns = ['Feature 1', 'Feature 2', 'Correlation']

alt.Chart(correlation_long).mark_rect().encode(
    x='Feature 1:0',
    y='Feature 2:0',
    color=alt.Color('Correlation:Q', scale=alt.Scale(scheme='viridis')),
    tooltip=['Feature 1', 'Feature 2', 'Correlation']
).properties(**plot_size,
    title="Correlation Heatmap"
)

```

Out[17]:



Discuss Correlation Plot:

```

In [18]: # This import is not working
# Getting the Following Error:
# AttributeError: `np.Inf` was removed in the NumPy 2.0 release. Use `np.inf`
from deepchecks.tabular.checks import FeatureLabelCorrelation, FeatureFeatureCorrelation
from deepchecks.tabular import Dataset

# Adapted from https://github.com/skysheng7/DSCI522_data_validation_demo/tree/master
# temp_train_ds = Dataset(train_df, label="class", cat_features=[])

# check_feat_lab_corr = FeatureLabelCorrelation().add_condition_feature_pps_
# check_feat_lab_corr_result = check_feat_lab_corr.run(dataset=temp_train_ds)

# check_feat_feat_corr = FeatureFeatureCorrelation().add_condition_max_number_of_features_
# check_feat_feat_corr_result = check_feat_feat_corr.run(dataset=temp_train_

```

```
# if not check_feat_lab_corr_result.passed_conditions():
#     raise ValueError("Feature-Label correlation exceeds the maximum accept

# if not check_feat_feat_corr_result.passed_conditions():
#     raise ValueError("Feature-feature correlation exceeds the maximum acce
```

/Users/jacob/miniforge3/envs/climate-env/lib/python3.11/site-packages/deephchecks/core/serialization/dataframe/html.py:16: UserWarning:

pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.

- [] 11. No anomalous correlations between features/explanatory variables

In [19]: *#region Pseudo Code:*

```
# definitely only use train dataset
# correlation plots on train_df across explanatory variables
# expectation is that there is no correlation between features unless
# they represent the same underlying change like day of year and month

#endregion

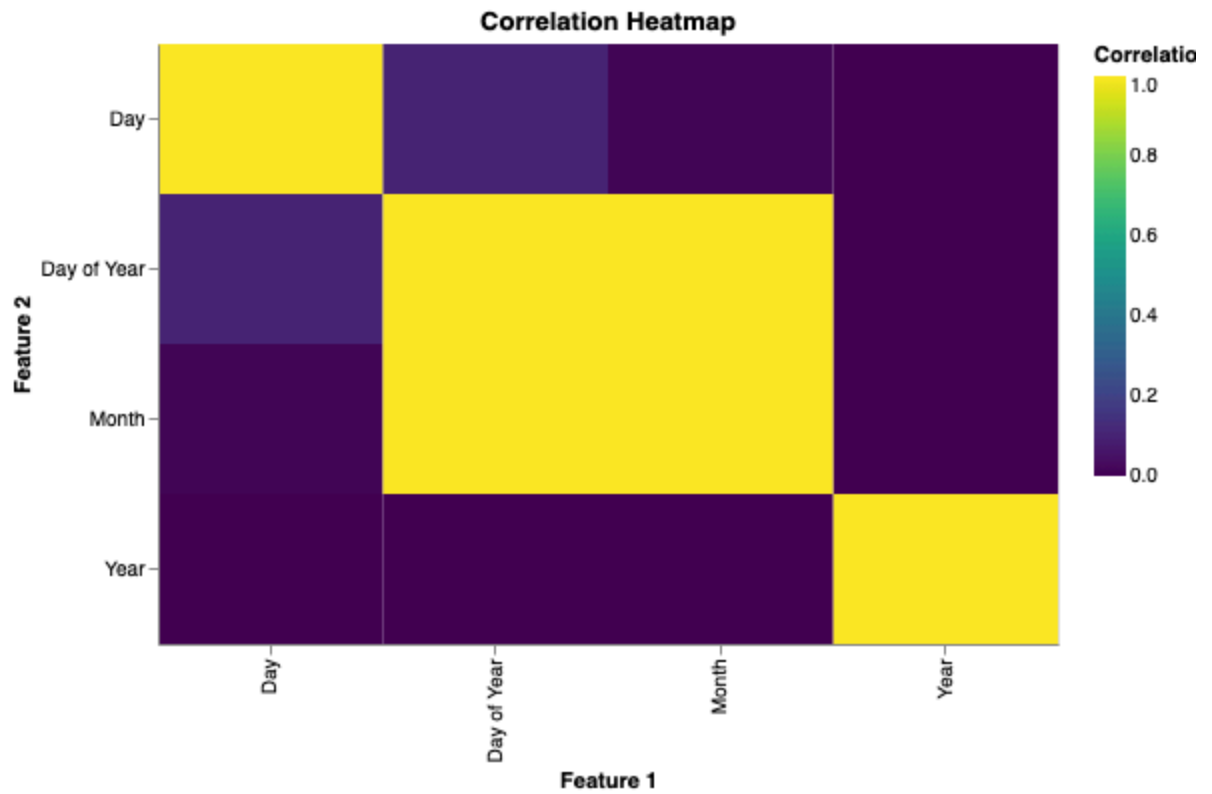
# Adapted from https://github.com/skysheng7/DSCI522_data_validation_demo/tree

# Correlation plot with Features only, train_df

correlation_matrix = train_df.drop(columns=['Month_Name', 'Anomaly', 'Temperature'])
correlation_long = correlation_matrix.reset_index().melt(id_vars='index')
correlation_long.columns = ['Feature 1', 'Feature 2', 'Correlation']

alt.Chart(correlation_long).mark_rect().encode(
    x='Feature 1:0',
    y='Feature 2:0',
    color=alt.Color('Correlation:Q', scale=alt.Scale(scheme='viridis')),
    tooltip=['Feature 1', 'Feature 2', 'Correlation']
).properties(**plot_size,
    title="Correlation Heatmap"
)
```

Out[19]:

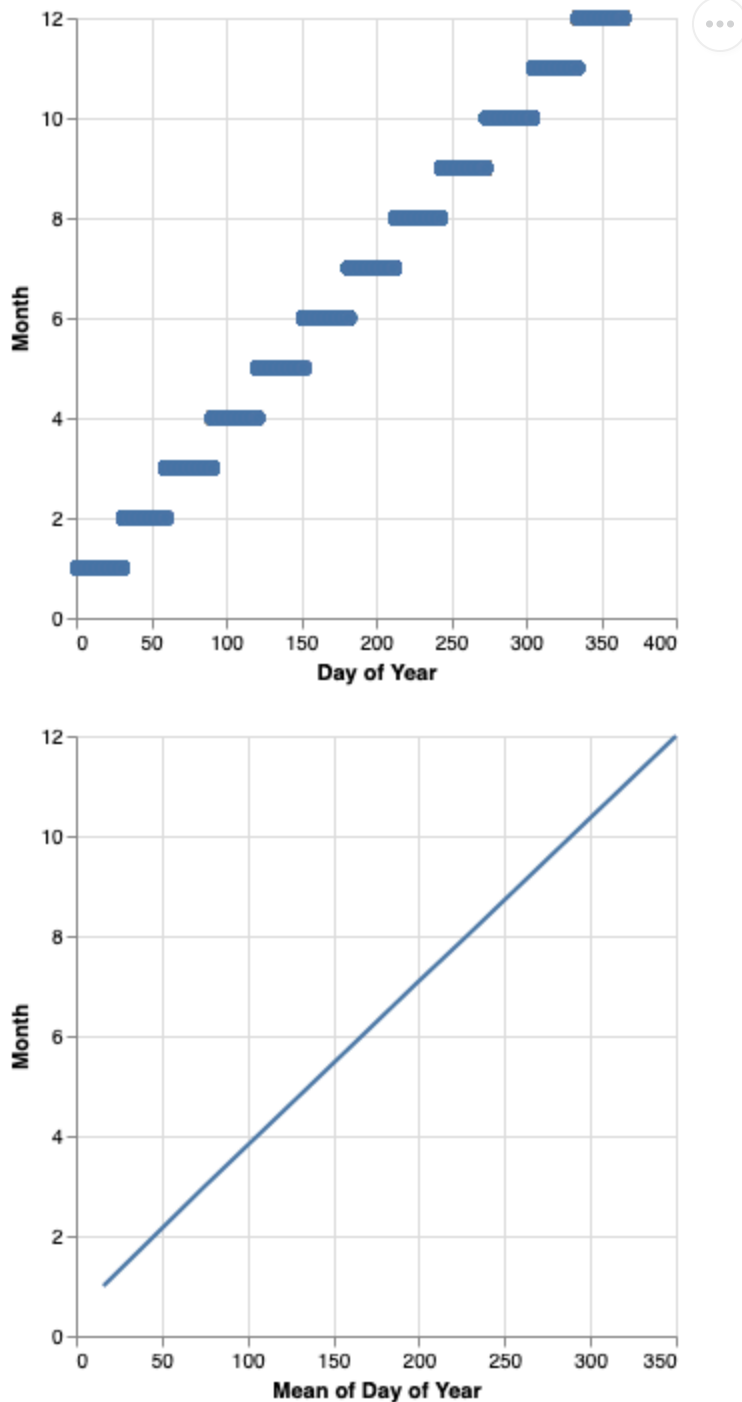


Discuss Correlation Plot:

In []:

```
In [20]: # Visualize why Day of Year
# and Month are Correlated
alt.Chart(train_df).mark_point().encode(
    x = 'Day of Year',
    y = 'Month'
)& alt.Chart(train_df).mark_line().encode(
    x = 'mean(Day of Year)',
    y = 'Month'
) # Hence high correlation due to linear relation
```

Out[20]:



Exploratory Data Analysis (EDA)

The goal of this EDA is to determine what type of predictive model will be the best fit for the data. A suitable regression method is to be explored, as the target (global average land temperature) is continuous. After preprocessing the dataset, no presence of null values were found requiring attention. In order to make the Month_Name (converting from Month) feature more readable during EDA, the feature was converted to an ordinal feature, with Jan as the first in the order and December as the last in the order. Monthly trends in the data were considered over the years, however overall trends were found

irrespective of the month the data was collected in. A cutoff year was decided for splitting the data into test and training data sets (see code below), and EDA was carried out on only the training portion of the data set to avoid violating the golden rule and double dipping. A randomized test split was not implemented as the model is desired for predicting temperatures in the future, so test data taken from the latest measurements represents the best evaluation of the regression model. Once the dataset was split, EDA was performed with several visualization strategies, the most informative of which are presented below. Discussions of the findings and rationale are found below as well.

```
In [21]: # Quick view of data and columns
train_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 48578 entries, 0 to 48577
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Year             48578 non-null  int64
1   Month            48578 non-null  int64
2   Day              48578 non-null  int64
3   Day of Year      48578 non-null  int64
4   Anomaly          48578 non-null  float64
5   Temperature      48578 non-null  float64
6   Month_Name       48578 non-null  category
dtypes: category(1), float64(2), int64(4)
memory usage: 2.6 MB
```

```
In [22]: # Overview of statistics
train_df.describe()
```

```
Out[22]:
```

	Year	Month	Day	Day of Year	Anomaly	Temperature
count	48578.000000	48578.000000	48578.000000	48578.000000	48578.000000	48578.000000
mean	1946.000947	6.522953	15.729569	183.000000	0.016194	19.999999
std	38.393532	3.448732	8.800154	105.331318	0.607513	10.000000
min	1880.000000	1.000000	1.000000	1.000000	-2.728000	10.000000
25%	1913.000000	4.000000	8.000000	92.000000	-0.367000	10.000000
50%	1946.000000	7.000000	16.000000	183.000000	-0.034000	10.000000
75%	1979.000000	10.000000	23.000000	274.000000	0.372750	10.000000
max	2012.000000	12.000000	31.000000	365.000000	2.552000	10.000000

```
In [23]: # Verify no null values
train_df.isna().any()
```

```
Out[23]: Year          False
Month          False
Day            False
Day of Year    False
Anomaly        False
Temperature    False
Month_Name     False
dtype: bool
```

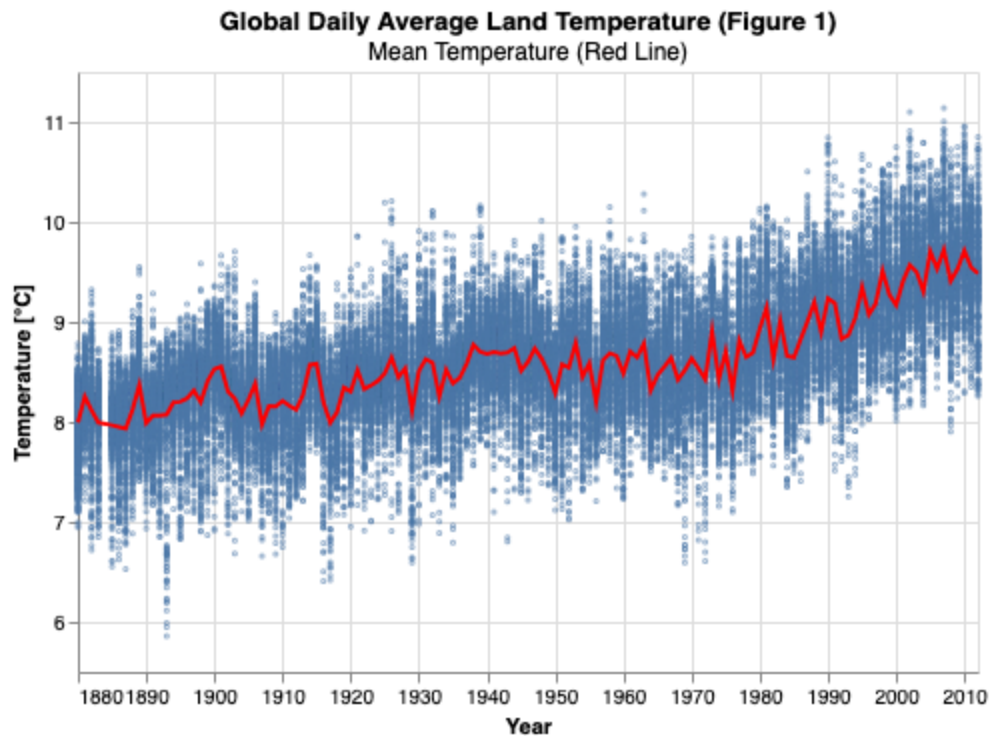
Due to the many measurements that occur in each year (contributing to plotting noise), the mean of the temperatures for each group of measurements taken in a year were plotted in addition to the raw data points of temperature over time. A general trend of increasing temperature over time with local fluctuations can be observed below.

```
In [24]: # Create scatter of raw data with some opacity to reduce plot noise
temp_points = alt.Chart(train_df,
                        title=alt.Title(
                            text='Global Daily Average Land Temperature (Figure 1)',
                            subtitle='Mean Temperature (Red Line)'
                        ).mark_point(opacity=0.6, size=1).encode(
                            x = 'Year:T',
                            y = 'Temperature:Q')

# Create line plot of the mean of all the measurements in a given year
temp_line_mean = temp_points.mark_line(size=2, color='red').encode(
    x = alt.X('Year:T', title='Year'),
    y = alt.Y('mean(Temperature):Q', title='Temperature [°C]'
              ).scale(zero=False)
).properties(**plot_size)

# show the raw data distribution along with mean by year
temp_points+temp_line_mean
```


Out [24]:



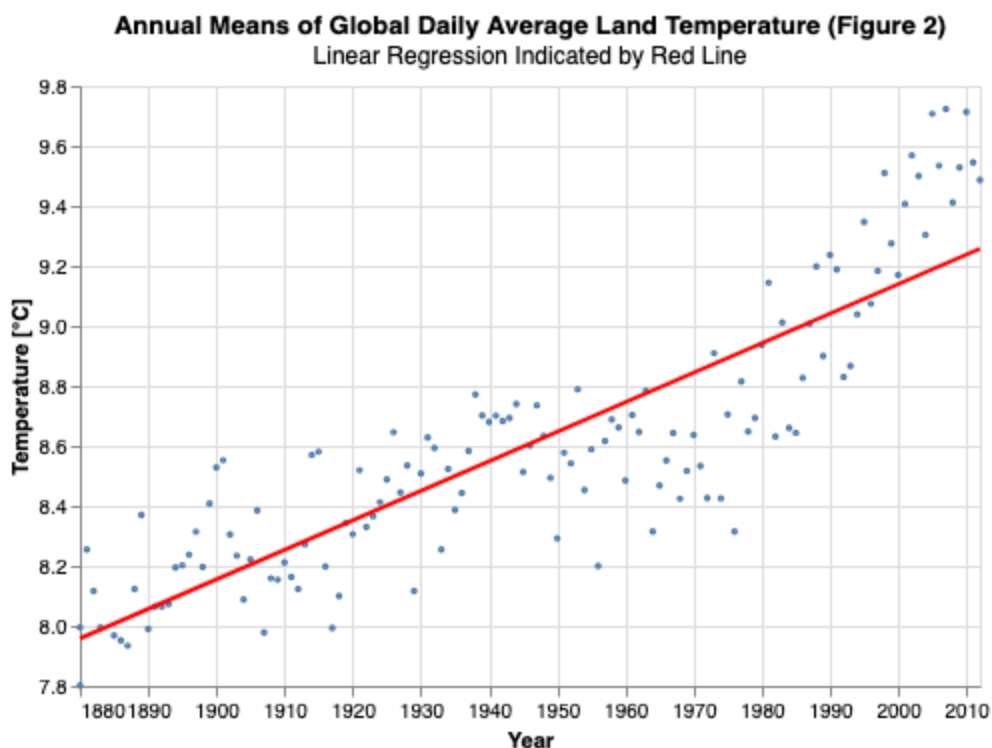
Using the mean data points by year, a linear regression model was fit to the training data to assess the viability of this approach in prediction. The linear regression fit can be seen to follow the overall trend, but misses information about local fluctuations, seen by the points above and below the line. A linear model has potential to generalize for unseen year examples but may be too simple for this analysis and will not be precise in its predictions for every year. A 5-year rolling average was also explored, due to its ability to adapt the curve more effectively to local fluctuations. However, a model of this type may not extrapolate well for unseen examples to predict future temperatures.

```
In [25]: # Plot a scatter plot of the mean temperatures for each year
temp_points_avg = alt.Chart(train_df,
                             title=alt.Title(
                                 text='Annual Means of Global Daily Average Land Temperature (Figure
                                     subtitle='Linear Regression Indicated by Red Line')
                             ).mark_point(size=2).encode(
                                 x = alt.X('Year:T', title='Year'),
                                 y = alt.Y('mean(Temperature):Q',
                                     title='Temperature [°C]').scale(zero=False)
                             )

# Add the regression line to the scatter plot and properties
reg = temp_points_avg+temp_points_avg.mark_line(
    size=2, color='red').transform_regression(
        'Year',
        'Temperature'
    ).properties(**plot_size)

# Show the regression plot
reg
```

Out [25]:



In order to check if the global average daily temperature over time was affected by seasonal phenomena, the annual means of these temperature measurements were analyzed by each month separated. It can be seen from the facet plot below that regardless of the season global daily average land temperature measurements are increasing non-trivially over time.

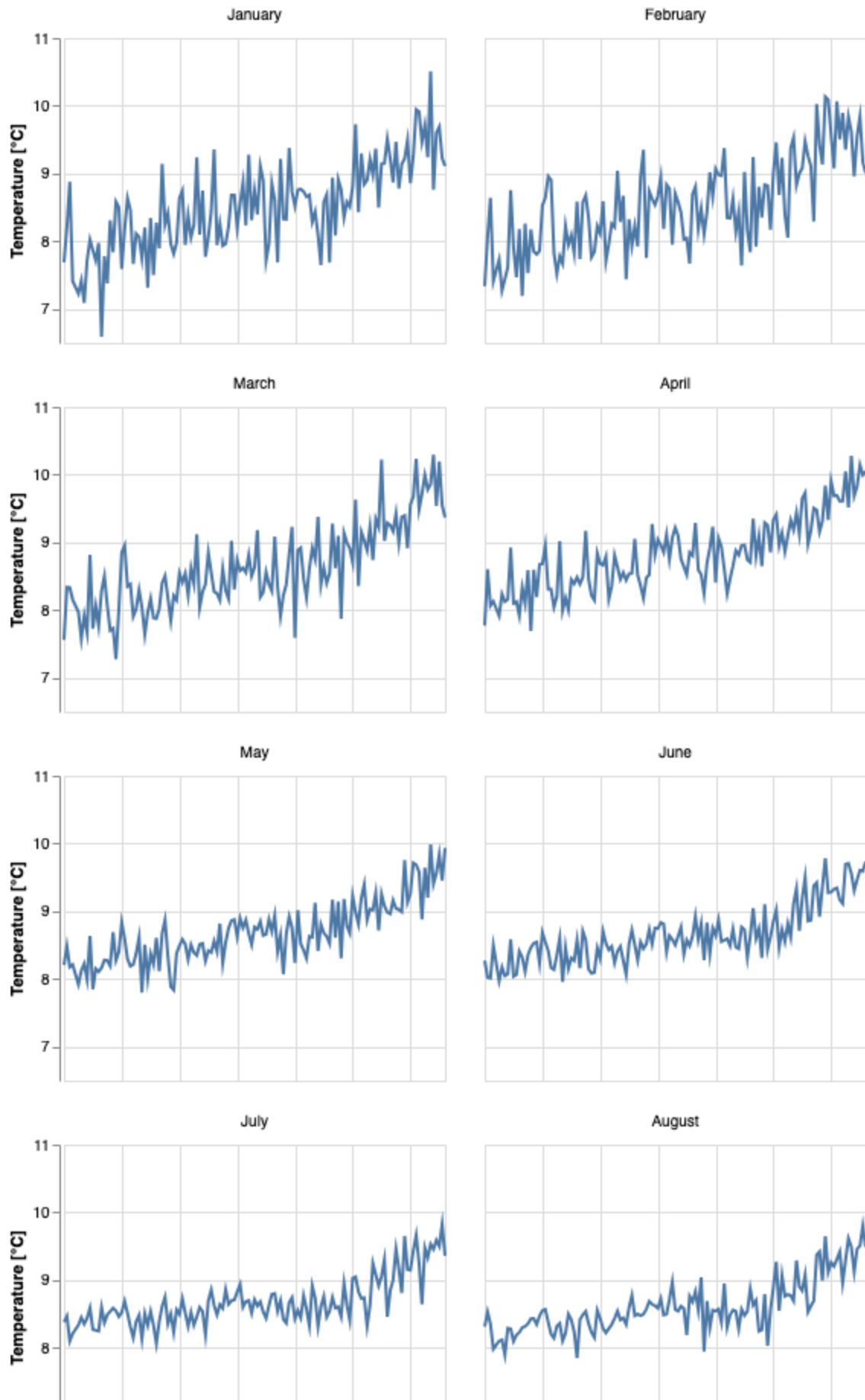
```
In [26]: # Average by year for each month in data
mean_per_month = train_df.groupby(
    ['Year', 'Month_Name'], observed=True)['Temperature'].mean().reset_index()

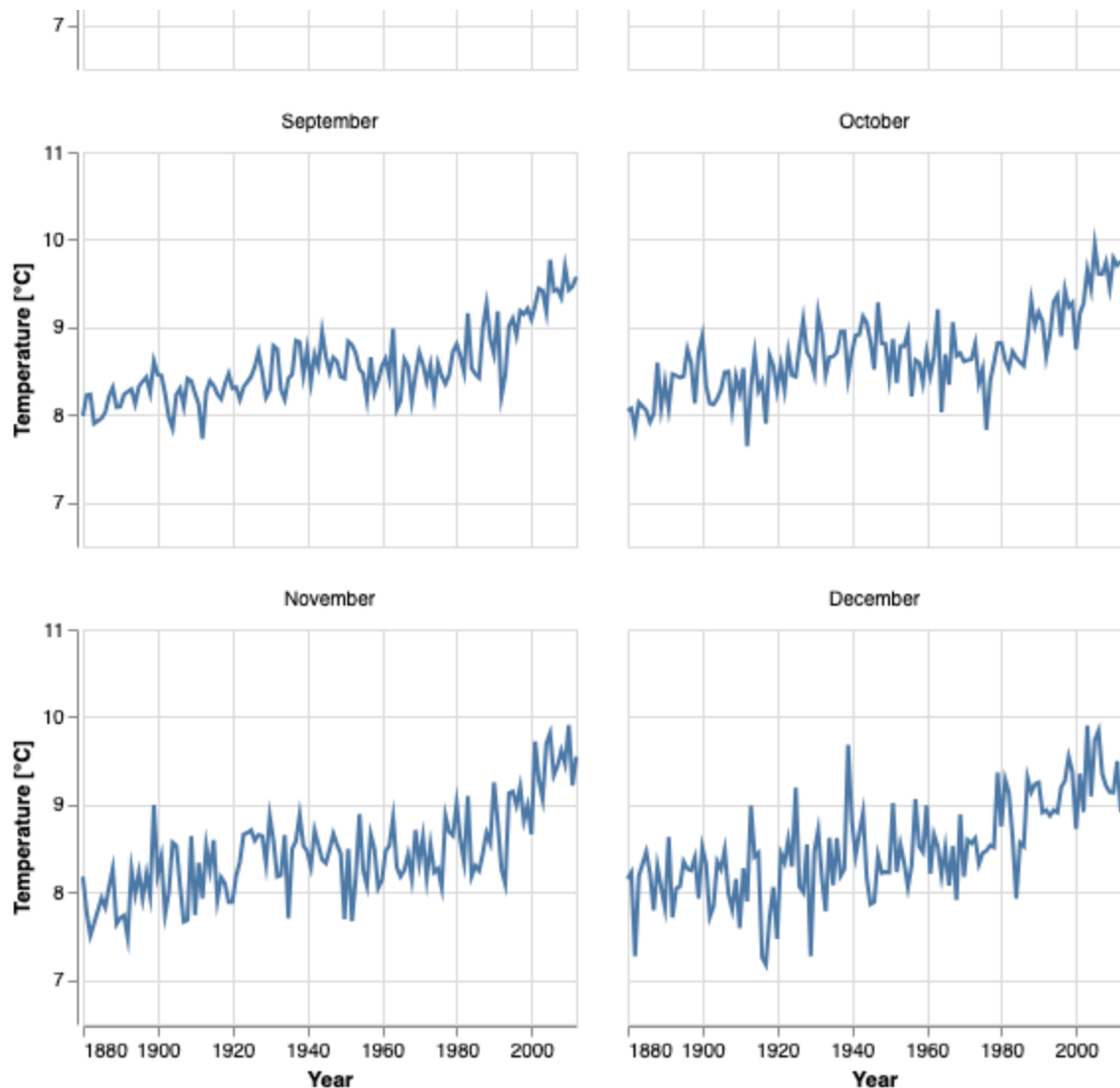
# For empty plotting title
mean_per_month[' '] = mean_per_month['Month_Name']

# Create Mean temperature plots and facet by month
temp_plot = alt.Chart(mean_per_month).mark_line().encode(
    x = 'Year:T',
    y = alt.Y('Temperature:Q', title='Temperature [°C]').scale(zero=False)
).properties(**facet_plot_size).facet(' ', columns=2)

# Show the plot with overall title
final_figure = alt.hconcat(temp_plot).properties(
    title='Seasonality of Annual Means of Global Daily Average Land Temperat
final_figure
```

Out[26]: **Seasonality of Annual Means of Global Daily Average Land Temperature (Figure 3)**





Finally, the distribution of the training data for temperature with respect to distinct year separated in by approximately 60 years were evaluated for distinct eras in the density plot below. The last year in the training data set (the cutoff year defined above) can be seen as significantly shifted to the right, indicating the increase in global daily average land temperature across time. Three distinct peaks in the density plot below reinforces the hypothesis that even when account for the variance in temperature due to local fluctuations, the overall trend of global temperature increasing over time remains.

```
In [27]: # Years to be analyzed separated by ~60 years
years_selection = [1880, 1960, cutoff]

# Select necessary data only
data_subset = train_df[train_df['Year'].isin(years_selection)]

# Create density plot for each selected year and add opacity to view overlap
temp_density = alt.Chart(data_subset
    ).transform_density(
        'Temperature',
        groupby=['Year'],
        as_=['Temperature', 'density']
```

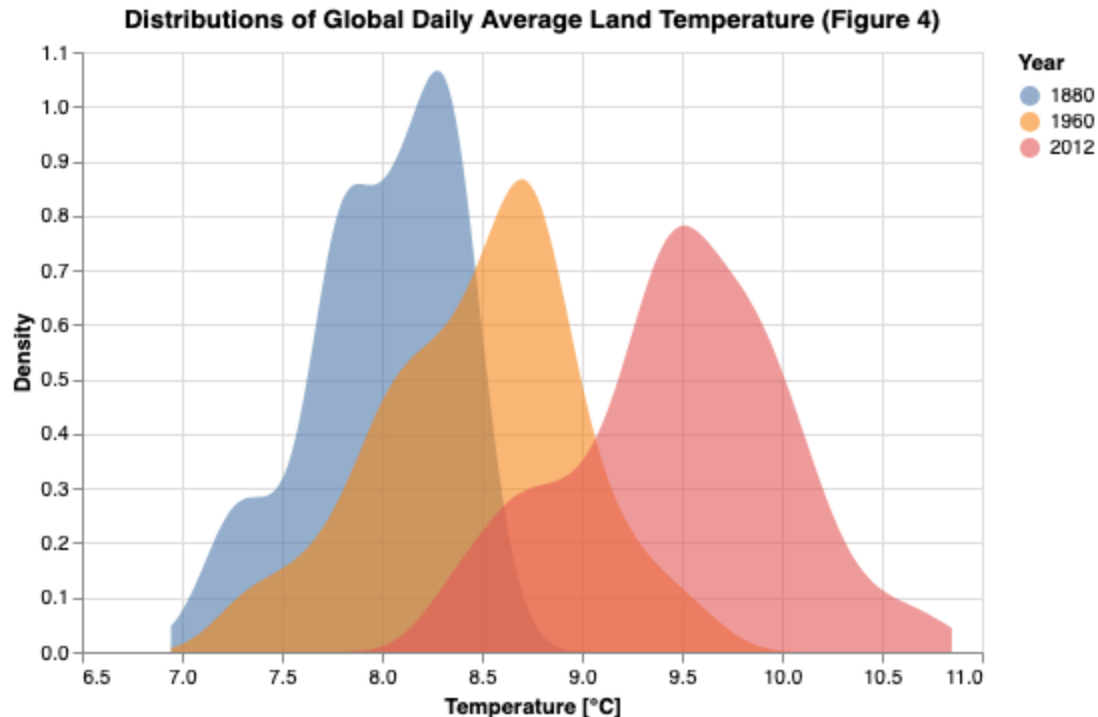
```

).mark_area(opacity=0.6).encode(
    x=alt.X('Temperature', title='Temperature [°C]'),
    y=alt.Y('density:Q', title='Density').stack(False),
    color = 'Year:N'
).properties(**plot_size, title = 'Distributions of Global Daily Average Land Temperature')

# Display the density plots by select years
temp_density

```

Out[27]:



Machine Learning Modelling

Here we convert daily climate data into yearly averages by creating a proper datetime index, resampling the anomaly values to annual means, and preparing the data for modeling by extracting the numerical year and separating it into feature (X) and target (y) arrays. Yearly averages smooth out short-term noise and make long-term climate trends easier to model, and using the year as the feature allows the machine-learning models to learn how temperature anomalies evolve over time.

```

In [28]: # -----
# 1. Convert daily data → yearly averages
# -----

# Note we are using the pre-split data for initial ML preprocessing, then wi

# Combine Year-Month-Day into a proper datetime column
df['Date'] = pd.to_datetime(df[['Year', 'Month', 'Day']])

# Use Date as index so resampling works correctly
df.set_index('Date', inplace=True)

```

```
# Resample to yearly means (YE = year-end)
yearly = df['Anomaly'].resample('YE').mean().reset_index()

# Extract year number from datetime column
yearly['Year'] = yearly['Date'].dt.year

# Feature matrix X (year), target vector y (anomaly)
X = yearly[['Year']].values
y = yearly['Anomaly'].values
```

We split the data into a training set (years up to 2012) and a test set (years after 2012) by creating boolean masks based on the chosen split year. It then uses those masks to separate the feature matrix and target values into training and testing subsets. By training on earlier years and testing on later years, we evaluate how well the model predicts future climate patterns rather than just fitting past data. A good model should perform well on the test set, meaning it can generalize to years it has never seen before.

```
In [29]: # -----
# 2. Train-Test Split (train: up to 2012, test: after 2012)
# -----

# note: same cutoff as used in previous splitting operation

train_mask = yearly['Year'] <= cutoff
test_mask  = yearly['Year'] > cutoff

X_train, y_train = X[train_mask], y[train_mask]
X_test,  y_test  = X[test_mask], y[test_mask]
```

Three different machine-learning models are defined which are Linear Regression, Random Forest, and a Support Vector Regressor (SVR) with scaling—so we can compare how well each one predicts yearly temperature anomalies. Each model captures patterns differently, ranging from simple linear trends to more flexible non-linear relationships.

```
In [30]: # -----
# 3. Define Models
# -----

models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(n_estimators=200, random_state=42),
    "SVR": Pipeline([
        ('scaler', StandardScaler()),
        ('svr', SVR(kernel='rbf', C=100, gamma=0.1, epsilon=0.01))
    ])
}
```

We train each defined model on the training data, predicts anomalies for the test years, and calculates key evaluation metrics (RMSE, MAE, R^2) to measure prediction accuracy. The results are stored in a table for easy comparison of model performance.

```
In [31]: # -----
# 4. TRAIN, PREDICT & EVALUATE (Output results as table)
# -----

results = {}

for name, model in models.items():
    model.fit(X_train, y_train)                # Train
    y_pred = model.predict(X_test)             # Predict
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)              # Evaluate

    results[name] = {"RMSE": rmse, "MAE": mae, "R2": r2}

# Convert results dictionary → nice DataFrame table
results_table = pd.DataFrame(results).T
print("\nModel Performance on Test Set:\n")
print(results_table)
```

Model Performance on Test Set:

	RMSE	MAE	R2
Linear Regression	0.539126	0.521939	-11.652955
Random Forest	0.350576	0.316120	-4.350267
SVR	0.145067	0.120141	0.083892

Lower RMSE and MAE values indicate more accurate predictions, while a higher (positive) R^2 shows the model explains more variance in the data. From the table, SVR has the lowest errors and a positive R^2 , making it the best model to use for forecasting.

```
In [32]: # -----
# 5. Select Best Model (based on RMSE)
# -----

best_model_name = results_table["RMSE"].idxmin()
best_model = models[best_model_name]
print(f"\nBest model: {best_model_name}")
```

Best model: SVR

We use the selected best model (SVR) to predict the temperature anomaly for the year 2030 and then adds it to the baseline temperature to estimate the actual land-average temperature.

```
In [33]: # -----
# 6. Forecast Temperature Anomaly for 2030
# -----

year_2030 = np.array([[2030]])
forecast_2030_anomaly = best_model.predict(year_2030)[0]

forecast_2030_temp = BASELINE_TEMP + forecast_2030_anomaly
```

```
print(f"\nPredicted anomaly for 2030: {forecast_2030_anomaly:.4f} °C")
print(f"Predicted land-average temperature for 2030: {forecast_2030_temp:.4f
```

Predicted anomaly for 2030: 1.9696 °C

Predicted land-average temperature for 2030: 10.5596 °C

The predicted anomaly of $\approx 1.97^{\circ}\text{C}$ indicates that 2030 is expected to be nearly 2°C warmer than the baseline period. Adding this to the baseline gives a land-average temperature of $\approx 10.56^{\circ}\text{C}$, showing a continuation of the observed warming trend.

```
In [34]: # -----
# 7. PLOT RESULTS
# -----

plt.figure(figsize=(10,6))

# Plot TRAIN data (blue)
plt.scatter(
    yearly['Year'][train_mask],
    yearly['Anomaly'][train_mask],
    alpha=0.6,
    color='blue',
    label="Train Data"
)

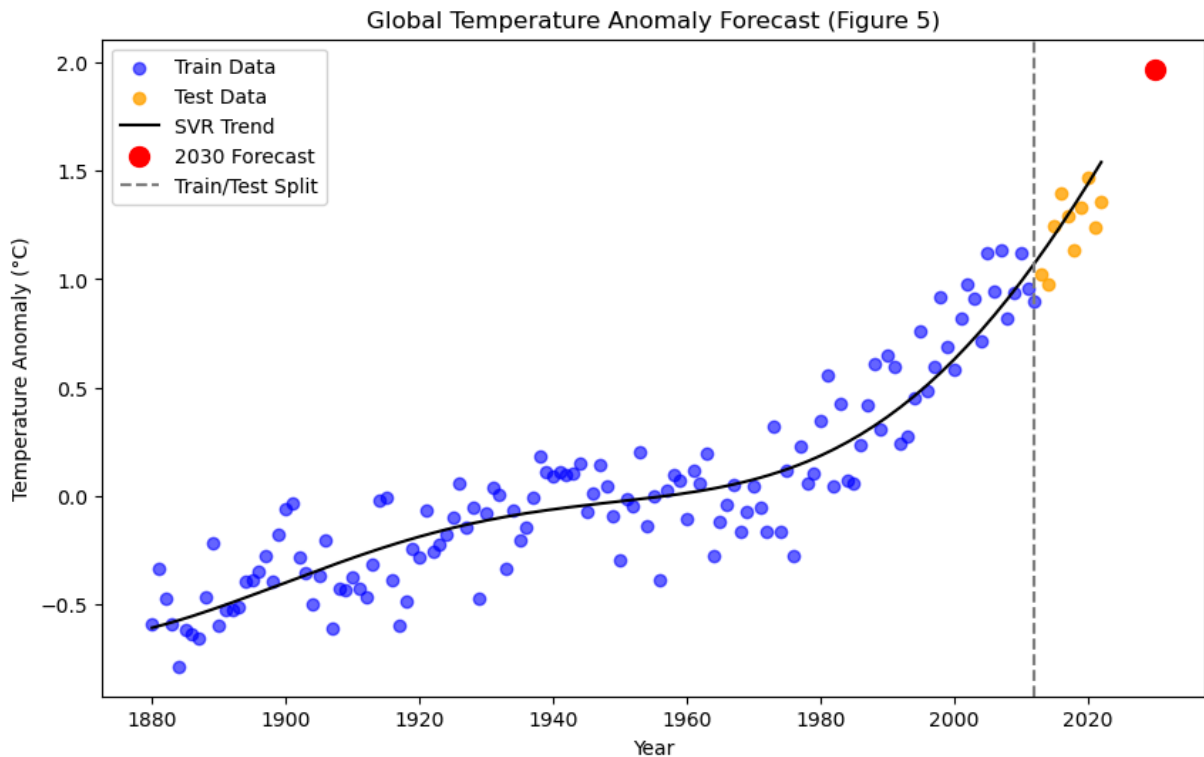
# Plot TEST data (orange/red)
plt.scatter(
    yearly['Year'][test_mask],
    yearly['Anomaly'][test_mask],
    alpha=0.8,
    color='orange',
    label="Test Data"
)

# Model trend line
plt.plot(yearly['Year'], best_model.predict(X), color='black', label=f"{best

# 2030 Forecast point
plt.scatter(2030, forecast_2030_anomaly, color='red', s=100, label="2030 For

# Train-test split line
plt.axvline(cutoff, color='gray', linestyle='--', label="Train/Test Split")

plt.xlabel("Year")
plt.ylabel("Temperature Anomaly (°C)")
plt.title("Global Temperature Anomaly Forecast (Figure 5)")
plt.legend()
plt.show()
```

The plot shows the historical anomalies, with training data in blue and test data in orange, allowing us to visually compare the model's predictions against unseen years. The black trend line from the SVR model captures the warming pattern, while the red point highlights the 2030 forecast, illustrating the expected continuation of the temperature rise.

The trend line closely follows historical data, test points are reasonably well-predicted, and the red forecast indicates that temperature anomalies are expected to continue rising, consistent with expert opinions on global warming.

Discussion

We found that global daily average land temperature has been steadily increasing since 1880, with a steeper increase after 1960. This trend is approximately the same across all months and seasons. The best model to capture this trend was a SVR model, which gave a predicted 2030 average land temperature of 10.6 C, an almost 2 C increase from the baseline period.

This is an expected result. Our result aligns well with expert opinions and well-documented patterns of climate change.

The impact of these findings is that they reinforce the global scientific consensus of continued warming of the earth. In future, we would recommend creating more models with additional features to see what features most impact the model and predict warming. For example, could incorporating CO2 levels improve these predictions?

Additionally, this model focuses on global averages, but we could use a more granular dataset to investigate if some parts of the world are warming faster than others.

References

Lindsey, R., & Dahlman, L. (2024, January 18). Climate change: Global temperature. NOAA Climate.gov. <https://www.climate.gov/news-features/understanding-climate/climate-change-global-temperature>

Intergovernmental Panel on Climate Change. (2018). Summary for policymakers: Global warming of 1.5 °C. In Global warming of 1.5 °C: An IPCC Special Report on the impacts of global warming of 1.5 °C above pre-industrial levels and related global greenhouse gas emission pathways. <https://www.ipcc.ch/sr15/chapter/spm/>

NASA. (n.d.). Climate change: Evidence. <https://science.nasa.gov/climate-change/evidence/>

National Centers for Environmental Information. (n.d.). Did you know? Anomalies vs. temperature. <https://www.ncei.noaa.gov/access/monitoring/dyk/anomalies-vs-temperature>